Find Loop

AlgoExpert · Hard · Linked Lists

The problem

- The list contains a cycle; return the node where the loop begins (Floyd's algorithm).

Intuition

- Floyd's cycle detection: after slow and fast meet, a pointer from the head and one from the meeting point meet at the loop start.

Approach

- 1. Slow moves one, fast two
- 2. They meet inside the loop
- 3. Reset one to the head
- 4. Move both by one until they meet

Java solution

```java
public class FindLoop {
    public static LinkedListNode findLoop(LinkedListNode head) {
        LinkedListNode slow = head.next, fast = head.next.next;
        while (slow != fast) { slow = slow.next; fast = fast.next.next; }
        slow = head;
        while (slow != fast) { slow = slow.next; fast = fast.next; }
        return slow;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Recap

- Floyd's cycle detection: after slow and fast meet, a pointer from the head and one from the meeting point meet at the loop start.
- Complexity: O(n)